

Спецификация требований к программному средству (SRS)

Программное средство автоматизации складской логистики на основе технологии RPA (WMS)

1. Введение

1.1 Назначение документа

Настоящий документ представляет собой техническое задание и спецификацию требований для команды разработчиков (Backend, Frontend, RPA-инженеры, QA). Документ описывает архитектуру, функциональные и нефункциональные требования, модели данных и стандарты кодирования для реализации WMS-системы с элементами роботизированной автоматизации процессов (RPA).

1.2 Описание продукта

Разрабатываемое программное средство — это облачная (веб) WMS-платформа для средних и крупных предприятий Республики Беларусь.

Ключевая особенность: использование программных RPA-роботов для бесшовной интеграции с закрытыми (legacy) ERP-системами без API и автоматического формирования строгой бухгалтерской/складской документации (ТТН, ТН, Акты).

Бизнес-модель: SaaS (Freemium).

2. Архитектура и стек технологий

Система проектируется на базе микросервисной архитектуры с использованием паттернов **Clean Architecture**, **DDD** (Domain-Driven Design), **CQRS**, **Event Sourcing** и **Saga** для распределенных транзакций.

2.1 Технологический стек

- **Frontend:** React (SPA), JavaScript/TypeScript.
- **Backend:** Java, Spring Boot, Spring Cloud (API Gateway), Spring Security.
- **Базы данных:** PostgreSQL (реляционная модель, каждая БД в 3НФ), Redis (кэширование сессий/данных).
- **Брокер сообщений:** RabbitMQ (Topic exchanges для доменных событий).
- **Обсервируемость:** OpenTelemetry, Jaeger, Prometheus, Grafana.

- **Документация API:** Swagger / OpenAPI 3.0.

2.2 Микросервисы (C4 Component Level)

В соответствии с принципом *Database-per-Service*, система разделена на 5 изолированных микросервисов:

1. **SSOService:** Аутентификация, авторизация (JWT), управление профилями и ролями. Интеграция с OAuth2 (Google/Yandex). БД: UserDB.
2. **OrganizationService:** Управление юридическими лицами (названия, УНП, адреса, статусы). БД: OrganizationDB.
3. **WarehouseService:** Управление топологией склада (склады, стеллажи: ячейки, полки, паллеты, холодильники с температурным режимом). БД: WarehouseDB.
4. **ProductService:** Складской учет. Ведение номенклатуры, справочник поставщиков, учет партий (batch), инвентаризация, отслеживание остатков. БД: ProductDB.
5. **DocumentsService:** Генерация и маршрутизация документов (приходные/отпускные ордера, акты списания/переоценки, описи). Включает модуль RPAExecutor. Не имеет собственной БД (использует данные событий).

3. Ролевая модель

Система поддерживает жесткое разграничение прав доступа (RBAC).

- **Заведующий складом (Manager):** Управление организацией, топологией склада, сотрудниками, доступ к аналитике и отчетам.
- **Кладовщик (Storekeeper):** Приемка, размещение (ручное/автоматическое), отгрузка, комплектация сборочных листов, формирование ордеров.
- **Бухгалтер (Accountant):** Проведение инвентаризации, переоценка (изменение стоимости), списание ТМЦ, формирование актов.
- **Сотрудник склада (Worker):** Линейный исполнитель (использует мобильные сканеры штрихкодов для фактического размещения/сборки).

4. Функциональные требования (Use Cases)

4.1 Модуль авторизации и управления персоналом

- **FR-1.1:** Система должна обеспечивать аутентификацию по логину (email)/паролю и через OAuth2.
- **FR-1.2:** Система должна поддерживать CRUD-операции для сотрудников (Заведующий складом может добавлять, изменять, блокировать аккаунты).
- **FR-1.3:** Аудит логинов. Каждая попытка входа логируется в таблицу user_login_audit (IP, User-Agent, время).

4.2 Модуль управления организацией и топологией склада

- **FR-2.1:** Ввод реквизитов организации (Название, УНП, Адрес). Валидация УНП на уникальность.
- **FR-2.2:** Создание складов и привязка их к организации.
- **FR-2.3:** Настройка топологии. Пользователь может создавать стеллажи различных типов (SHELF, CELL, FRIDGE, PALLET), задавая их габариты (длина, ширина, высота) и ограничения (грузоподъемность, температурный режим).

4.3 Модуль логистических операций (Приемка, Размещение, Отгрузка)

- **FR-3.1 Приемка:** Регистрация поставки, создание карточек товаров (Артикул, Штрих-код, Срок годности).
- **FR-3.2 Размещение (Ручное):** Валидация: Система должна запрещать размещение, если вес/габариты товара превышают лимиты ячейки (BR-3, BR-4), или нарушаются условия хранения температурного режима (BR-2).
- **FR-3.3 Размещение (Автоматическое):** Система должна предлагать ячейку на основе габаритов и алгоритмов ABC-анализа.
- **FR-3.4 Отгрузка:** При отборе товара система должна применять правило FIFO (учитывать дату поступления и сроки годности партии).
- **FR-3.5 Сканирование:** Обработка событий от сканеров штрих-кодов в реальном времени.

4.4 Модуль бухгалтерии (Инвентаризация, Переоценка, Списание)

- **FR-4.1 Инвентаризация:** Сохранение "среза" остатков на момент старта. Ввод фактических данных. Автоматический расчет разницы.
- **FR-4.2 Переоценка:** Блокировка ввода нулевой или отрицательной цены. Пересчет стоимости остатков партии.
- **FR-4.3 Списание:** Обнуление/уменьшение остатка с обязательным указанием причины (брак, истечение срока).

4.5 Модуль RPA и генерации документов

- **FR-5.1 RPA Extractor:** Автоматический фоновый опрос (раз в сутки) закрытых сторонних ERP систем заказчика для выгрузки плановых поставок.
- **FR-5.2 RPA Document Builder:** Автоматическое заполнение форм в стандартах РБ (ТТН, ТН, Инвойсы) в форматах .pdf, .xlsx, .docx.

5. Модель данных (Базы данных)

Во всех сервисах (кроме Documents) применяется принцип **Event Sourcing** для построения надежного аудиторского следа.

- **Read Models (Состояние):** Таблицы вида `entity_read_model` (например, `user_read_model`, `warehouse_read_model`) хранят текущее материализованное

состояние (CQRS - слой чтения).

- **Event Store (История):** Таблицы вида `entity_events` (например, `user_events`, `stock_event`) хранят JSONB-объекты всех изменений (`event_type`, `event_data`). Мутация Read-моделей происходит *только* как реакция на событие из Event Store.

6. Нефункциональные требования (NFR)

6.1 Производительность и надежность

- **PER-1:** Время отклика на UI-действия и стандартные запросы БД не должно превышать 5 секунд.
- **ROB-1:** Кэширование состояния UI (в Redis или LocalStorage) для защиты от разрывов сети на складе.
- **AVL-1:** Доступность системы 99.9% в рабочее время склада.

6.2 Безопасность

- **SEC-1:** JWT токены с временем жизни (TTL) строго 1 час. При отсутствии активности требуется перелогин.
- **SEC-2:** Пароли хранятся в БД строго в виде хэшей (реализация через `BCryptPasswordEncoder`).
- **SEC-3:** Все API эндпоинты (кроме `/api/auth/**`) должны быть защищены фильтром `JwtAuthenticationFilter`.

6.3 Пользовательский интерфейс (UX/UI)

- **Дизайн система:** Основные цвета — желтый (`#FFE673`) и синий (`#1976D2`). Семантические цвета (красный для ошибок, оранжевый - warning, зеленый - success).
- **Шрифты:** Основной шрифт — Gantari/Jost. Строгая иерархия H1-H4.
- **Адаптивность:** Интерфейс должен корректно отображаться на десктопах и планшетах/сканерах кладовщиков.

7. Требования к качеству кода и разработке

Разработка должна вестись в соответствии с принципами Clean Code.

7.1 Паттерны проектирования (Обязательные к использованию)

- **Builder:** Для генерации сложных DTO и доменных сущностей (через Lombok `@Builder`).
- **Adapter / ColumnTransformer:** Для маппинга PostgreSQL enum-типов (`user_role`, `auth_provider`) в Java Enum (через `@org.hibernate.annotations.ColumnTransformer`).
- **Strategy:** Для подмены алгоритмов (например, `PasswordEncoder`).
- **Chain of Responsibility:** Для конфигурации `SecurityFilterChain`.
- **Facade:** Сервисный слой (например, `ProfileService`) выступает фасадом,

оркестрирующим вызовы к нескольким репозиториям.

7.2 Принципы проектирования

- **SLAP** (Single Level of Abstraction Principle): Метод контроллера/сервиса не должен смешивать высокоуровневую бизнес-логику (проверки) и низкоуровневую (прямые SQL вызовы).
- **SOLID**: Четкое разделение интерфейсов (UserRepository абстрагирован от СУБД).
- **DRY**: Общая логика (работа с JWT, аудит) должна быть вынесена в общие утилитные/инфраструктурные сервисы.

7.3 Тестирование и документация

- **Swagger/OpenAPI**: Каждый контроллер обязан иметь аннотацию @Tag, каждый метод — @Operation, каждое DTO — @Schema с примерами (example) для удобного импорта в Postman.
- **Модульные тесты (Unit Tests)**: Покрытие бизнес-логики сервисов с использованием JUnit 5 и Mockito.
- **Интеграционные тесты**: Проверка взаимодействия микросервисов, поднятых через Testcontainers (RabbitMQ, PostgreSQL).
- **Code Style**: Проверка с помощью Checkstyle на этапе CI/CD пайплайна (сборка не должна проходить при наличии ошибок стиля).